

Questões de POO

Herança

Questão 1. Complete corretamente o código:

```
class Conta {  
    double saldo = 0;  
  
    void depositar(double valor) {  
        saldo += valor;  
    }  
}  
  
class ContaPoupanca _____ Conta {  
    void renderJuros() {  
        saldo += saldo * 0.01;  
    }  
}
```

- A) implements
- B) extends
- C) super
- D) with
- E) override

Questão 2. Observe o código:

```
class Conta {  
    double saldo = 100;  
  
    void sacar(double valor) {  
        saldo -= valor;  
    }  
}  
  
class ContaSalario extends Conta {  
    @override  
    void sacar(double valor) {  
        if (valor <= saldo) {  
            saldo -= valor;  
        }  
    }  
}  
  
void main() {  
    ContaSalario conta = ContaSalario();  
    conta.sacar(150);  
    print(conta.saldo);  
}
```

Qual será a saída?

- A) -50
- B) 0
- C) 100
- D) 150
- E) Erro de compilação.

Questão 3. Analise o código:

```
class Funcionario {  
    String nome;  
  
    Funcionario(this.nome);  
}  
  
class Professor extends Funcionario {  
    String disciplina;  
  
    Professor(this.disciplina);  
}
```

Qual erro existe no código?

- A) Professor não pode herdar de Funcionario.
- B) A classe filha precisa chamar o construtor da classe pai.
- C) O atributo disciplina deveria estar na classe pai.
- D) A classe Funcionario deveria ser abstrata.
- E) O atributo nome deveria ser double.

Questão 4. Qual alternativa corrige corretamente o código da questão anterior?

A)

```
class Professor extends Funcionario {  
    String disciplina;  
  
    Professor(String nome, this.disciplina) : super(nome);  
}
```

B)

```
class Professor extends Funcionario {  
    String disciplina;  
  
    Professor(this.disciplina) : this.nome;  
}
```

C)

```
class Professor extends Funcionario {  
    String disciplina;  
  
    Professor(this.disciplina) : Funcionario(nome);  
}
```

D)

```
class Professor extends Funcionario {  
    String disciplina;  
  
    Professor(this.disciplina) : extends(nome);  
}
```

E)

```
class Professor extends Funcionario {  
    String disciplina;  
  
    Professor(this.disciplina) : override(nome);  
}
```

Questão 5. Analise as afirmativas sobre herança:

- I. Uma classe filha pode herdar métodos da classe pai.
 - II. Uma classe filha pode sobrescrever métodos herdados.
 - III. Em Dart, uma classe pode herdar diretamente de apenas uma classe usando extends.
 - IV. Os construtores da classe pai são herdados automaticamente pela classe filha.
- A) Apenas I e II estão corretas.
B) Apenas I, II e III estão corretas.
C) Apenas II, III e IV estão corretas.
D) Apenas I e IV estão corretas.
E) Todas estão corretas.

Questão 6. Em Dart, qual é a forma correta de declarar que ContaCorrente herda de Conta?

- A) class ContaCorrente implements Conta {}
B) class ContaCorrente inherits Conta {}
C) class ContaCorrente extends Conta {}
D) class ContaCorrente : Conta {}
E) class ContaCorrente super Conta {}

Questão 7. Qual alternativa representa corretamente uma relação de herança do tipo “é um”?

- A) Motor extends Carro, pois motor é um tipo de carro.
B) Teclado extends Computador, pois teclado é um tipo de computador.
C) ContaCorrente extends Conta, pois conta corrente é um tipo de conta.
D) Endereco extends Pessoa, pois endereço é um tipo de pessoa.
E) Roda extends Bicicleta, pois roda é um tipo de bicicleta.

Questão 8. Analise o seguinte cenário:

Um sistema possui as classes Conta, ContaCorrente, ContaPoupanca e ContaSalario. Todas possuem saldo, depósito e saque. Porém, cada tipo de conta possui uma regra diferente para sacar dinheiro.

Qual é a melhor solução usando herança?

- A) Criar apenas uma classe Conta com vários if para descobrir o tipo da conta.
- B) Criar uma classe pai Conta com comportamentos comuns e sobrescrever o método sacar() nas classes filhas.
- C) Criar classes sem relação entre si, repetindo saldo, depósito e saque em todas.
- D) Criar a classe Conta herdando de ContaCorrente, ContaPoupanca e ContaSalario.
- E) Criar uma classe Saque herdando de todas as contas.

Classes abstratas

Questão 9. Qual alternativa corrige corretamente o código da questão anterior?

A)

```
class ContaCorrente extends Conta {  
    void sacar;  
}
```

B)

```
class ContaCorrente extends Conta {  
    @override  
    void sacar(double valor) {  
        saldo -= valor;  
    }  
}
```

C)

```
class ContaCorrente extends Conta {  
    abstract void sacar(double valor);  
}
```

D)

```
class ContaCorrente extends Conta {  
    sacar(valor);  
}
```

E)

```
class ContaCorrente extends Conta {  
    void Conta.sacar(double valor) {  
        saldo -= valor;  
    }  
}
```

Questão 10. Complete corretamente o código:

```
abstract class Poligono {  
    int quantidadeLados();  
  
    void desenhar() {  
        print('Desenhando polígono');  
    }  
}
```

```

    }
}

class Triangulo _____ Poligono {
    @override
    int quantidadeLados() {
        return 3;
    }
}

```

- A) implements
- B) abstract
- C) extends
- D) override
- E) super

Questão 11. Analise o código:

```

abstract class Funcionario {
    String nome;

    Funcionario(this.nome);

    double calcularSalario();
}

class Professor extends Funcionario {
    double valorHora;
    int horas;

    Professor(this.valorHora, this.horas);

    @override
    double calcularSalario() {
        return valorHora * horas;
    }
}

```

Qual erro existe no código?

- A) Uma classe abstrata não pode ter construtor.
- B) Professor precisa chamar o construtor da classe pai Funcionario.
- C) O método calcularSalario() não pode retornar double.
- D) Professor não pode sobrescrever método abstrato.
- E) A classe Funcionario não pode ter atributo.

Questão 12. Um sistema possui diferentes formas geométricas: Triangulo, Quadrado e Circulo. Todas devem calcular área, mas cada uma calcula de maneira diferente. Algumas também devem reaproveitar um método comum chamado exibirDescricao().

Qual é a melhor solução usando classe abstrata?

- A) Criar uma classe abstrata Forma com o método abstrato calcularArea() e o método concreto exibirDescricao().

- B) Criar apenas uma classe Forma com vários if para descobrir qual forma está sendo usada.
- C) Criar três classes sem relação entre si, repetindo todos os métodos.
- D) Criar Triangulo como classe pai de Quadrado e Circulo.
- E) Criar objetos diretamente da classe abstrata Forma.

Questão 13. Observe o código:

```
abstract class Animal {  
    void emitirSom();  
}  
  
class Cachorro extends Animal {  
    @override  
    void emitirSom() {  
        print('Au au');  
    }  
}  
  
void main() {  
    Cachorro c = Cachorro();  
    c.emitirSom();  
}
```

Qual será a saída?

- A) Animal
- B) Som genérico
- C) Au au
- D) Erro, pois classe abstrata não pode ter métodos.
- E) Erro, pois Cachorro não pode herdar de Animal.

Questão 14. Observe o código:

```
abstract class Forma {  
    double calcularArea();  
}  
  
void main() {  
    Forma f = Forma();  
}
```

Qual é o erro?

- A) A classe Forma não pode ter método abstrato.
- B) O método calcularArea() deveria retornar int.
- C) Uma classe abstrata não pode ser instanciada diretamente.
- D) A classe Forma deveria usar extends.
- E) O método main() não pode usar classes abstratas.

Questão 15. Qual alternativa declara corretamente uma classe abstrata em Dart?

A)

```
class abstract Animal {  
}
```

B)

```
abstract class Animal {  
}
```

C)

```
class Animal abstract {  
}
```

D)

```
interface Animal {  
}
```

E)

```
abstract Animal class {  
}
```

Questão 16. Analise as afirmativas:

I. Uma classe abstrata pode ter métodos concretos.

II. Uma classe abstrata pode ter métodos abstratos.

III. Uma classe abstrata pode ser instanciada diretamente.

IV. Uma classe concreta que herda uma classe abstrata deve implementar os métodos abstratos herdados.

A) Apenas I e II estão corretas.

B) Apenas I, II e IV estão corretas.

C) Apenas II e III estão corretas.

D) Apenas III e IV estão corretas.

E) Todas estão corretas.

Interfaces

Questão 17. Em Programação Orientada a Objetos, uma interface é:

A) Uma classe usada apenas para armazenar atributos privados.

B) Um contrato que define comportamentos que uma classe deve implementar.

C) Uma forma de impedir que uma classe tenha métodos.

D) Um tipo de construtor usado em classes filhas.

E) Um recurso usado apenas para criar objetos concretos.

Questão 18. Qual alternativa declara corretamente que OrdenacaoBolha implementa a interface Ordenador?

A)

```
class OrdenacaoBolha extends Ordenador {  
}
```

B)

```
class OrdenacaoBolha implements Ordenador {  
}
```

C)

```
class OrdenacaoBolha interface Ordenador {  
}
```

D)

```
class OrdenacaoBolha with Ordenador {  
}
```

E)

```
class OrdenacaoBolha override Ordenador {  
}
```

Questão 19. Observe o código:

```
abstract class Salvavel {  
    void salvar();  
}  
  
abstract class Exportavel {  
    void exportar();  
}  
  
class Documento implements Salvavel, Exportavel {  
    @override  
    void salvar() {  
        print('Salvando documento');  
    }  
}
```

Qual erro existe no código?

A) Documento não pode implementar duas interfaces.

B) Documento precisa implementar também o método exportar().

C) salvar() deveria retornar String.

D) Exportavel deveria herdar de Salvavel.

E) Interfaces não podem ter métodos.

Questão 20. Complete corretamente o código:

```
abstract class Pagamento {  
    void pagar(double valor);  
}  
  
class PagamentoPix _____ Pagamento {  
    @override  
    void pagar(double valor) {  
        print('Pagando via Pix: R$ $valor');  
    }  
}
```

- A) extends
- B) implements
- C) super
- D) override
- E) abstract

Questão 21. Observe o código:

```
abstract class Ordenador {  
    List<int> ordenar(List<int> numeros);  
}  
  
class OrdenacaoCrescente implements Ordenador {  
}
```

Qual é o problema do código?

- A) Uma classe não pode implementar uma interface em Dart.
- B) Ordenador deveria ser uma classe concreta.
- C) OrdenacaoCrescente precisa implementar o método ordenar().
- D) O método ordenar() deveria retornar void.
- E) A lista deveria ser obrigatoriamente List<double>.

Questão 22. Analise as afirmativas sobre interfaces:

- I. Uma interface define um contrato de métodos que uma classe deve implementar.
 - II. Uma classe pode implementar mais de uma interface.
 - III. Ao usar implements, a classe herda automaticamente o código pronto dos métodos da interface.
 - IV. Interfaces ajudam a reduzir o acoplamento entre classes.
- A) Apenas I e II estão corretas.
 - B) Apenas I, II e IV estão corretas.
 - C) Apenas II, III e IV estão corretas.
 - D) Apenas III e IV estão corretas.
 - E) Todas estão corretas.

Questão 23. Analise o código:

```
abstract class Animal {  
    void comer();  
}  
  
class Pessoa implements Animal {  
    @override  
    void comer() {  
        print('Pessoa comendo');  
    }  
}
```

Qual análise está correta?

- A) O código está incorreto, pois pessoa não pode implementar comportamento.
- B) O código está correto, pois uma interface pode representar um contrato de comportamento, como comer().
- C) O código está incorreto, pois implements só pode ser usado com classes concretas.
- D) O código está incorreto, pois toda interface precisa ter atributos.
- E) O código está correto apenas se Pessoa também tiver um método andar().

Questão 24. Em Dart, qual palavra-chave é usada para indicar que uma classe implementa uma interface?

- A) extends
- B) with
- C) implements
- D) interface
- E) override

Herança múltipla

Questão 25. Em Programação Orientada a Objetos, herança múltipla ocorre quando:

- A) Uma classe possui vários atributos.
- B) Uma classe herda diretamente de duas ou mais classes pai.
- C) Uma classe possui vários métodos com nomes diferentes.
- D) Uma classe implementa apenas um método abstrato.
- E) Uma classe cria vários objetos no main.

Questão 26. Em Dart, a herança múltipla de classes usando extends é:

- A) Permitida sem restrições.
- B) Permitida apenas para classes abstratas.
- C) Permitida apenas quando as classes pai não possuem atributos.
- D) Inválida, pois uma classe só pode usar extends com uma única classe pai.

E) Obrigatória quando se trabalha com interfaces.

Questão 27. Complete corretamente o código para evitar herança múltipla inválida em Dart:

```
abstract class Voador {  
    void voar();  
}  
  
abstract class Nadador {  
    void nadar();  
}  
  
class Pato _____ Voador, Nadador {  
    @override  
    void voar() {  
        print('Pato voando');  
    }  
  
    @override  
    void nadar() {  
        print('Pato nadando');  
    }  
}
```

- A) extends
- B) implements
- C) super
- D) override
- E) inherits

Questão 28. Analise o cenário:

Animal possui o método mover().

Mamifero herda de Animal e sobrescreve mover().

Ave herda de Animal e sobrescreve mover().

Morcego tenta herdar de Mamifero e Ave.

Qual problema conceitual pode ocorrer em uma linguagem com herança múltipla?

- A) Morcego não poderia ter métodos próprios.
- B) Morcego teria obrigatoriamente saldo bancário.
- C) Pode haver ambiguidade sobre qual versão de mover() deve ser herdada.
- D) Animal deixaria de ser uma classe.
- E) Ave e Mamifero seriam automaticamente interfaces.

Questão 29. Analise as afirmativas:

- I. Dart permite herança múltipla direta de classes usando extends.
- II. Dart permite implementar múltiplas interfaces usando implements.
- III. O problema do diamante está relacionado a ambiguidades de herança múltipla.

IV. Uma alternativa à herança múltipla pode ser o uso de composição.

- A) Apenas I e II estão corretas.
- B) Apenas II e III estão corretas.
- C) Apenas II, III e IV estão corretas.
- D) Apenas I, III e IV estão corretas.
- E) Todas estão corretas.

Questão 30. O “problema do diamante” ocorre quando:

- A) Uma classe não possui construtor.
- B) Uma classe filha herda o mesmo método por caminhos diferentes e a linguagem precisa decidir qual versão usar.
- C) Uma classe possui apenas métodos abstratos.
- D) Uma classe usa encapsulamento corretamente.
- E) Uma classe implementa apenas uma interface.

Polimorfismo

Questão 31. Complete corretamente o código para aplicar polimorfismo:

```
abstract class Ordenador {  
    List<int> ordenar(List<int> numeros);  
}  
  
class OrdenacaoCrescente implements Ordenador {  
    @Override  
    List<int> ordenar(List<int> numeros) {  
        numeros.sort();  
        return numeros;  
    }  
}  
  
void executarOrdenacao(_____ ordenador) {  
    print(ordenador.ordenar([3, 1, 2]));  
}
```

- A) OrdenacaoCrescente
- B) Ordenador
- C) List<int>
- D) int
- E) void

Questão 32. Analise o código:

```
class Relatorio {
    void gerar() {
        print('Gerando relatório');
    }
}

class RelatorioPDF extends Relatorio {
    @override
    void gerar() {
        print('Gerando PDF');
    }
}

class RelatorioExcel extends Relatorio {
    @override
    void gerar() {
        print('Gerando Excel');
    }
}

void main() {
    Relatorio r = RelatorioPDF();
    r = RelatorioExcel();
    r.gerar();
}
```

Qual será a saída?

- A) Gerando relatório
- B) Gerando PDF
- C) Gerando Excel
- D) Erro, pois a variável r não pode receber outro objeto.
- E) Erro, pois polimorfismo só funciona com classes abstratas.

Questão 33. Analise as afirmativas:

- I. Polimorfismo permite escrever código mais flexível.
 - II. Polimorfismo pode ocorrer com herança.
 - III. Polimorfismo pode ocorrer com interfaces.
 - IV. Polimorfismo exige obrigatoriamente herança múltipla.
- A) Apenas I e II estão corretas.
 - B) Apenas II e III estão corretas.
 - C) Apenas I, II e III estão corretas.
 - D) Apenas III e IV estão corretas.
 - E) Todas estão corretas.

Questão 34. Observe o código:

```
class Animal {  
    void emitirSom() {  
        print('Som genérico');  
    }  
}  
  
class Cachorro extends Animal {  
    void latir() {  
        print('Au au');  
    }  
}  
  
void main() {  
    Animal animal = Cachorro();  
    animal.latir();  
}
```

Qual é o erro?

- A) Cachorro não pode herdar de Animal.
- B) Animal deveria ser obrigatoriamente abstrata.
- C) A variável animal é do tipo Animal, e esse tipo não declara o método latir().
- D) O método latir() deveria ser chamado automaticamente.
- E) O método emitirSom() deveria retornar String.

Questão 35. Qual alternativa apresenta um exemplo de polimorfismo?

A)

```
int idade = 20;  
double altura = 1.75;
```

B)

```
class Pessoa {  
    String nome = '';  
}
```

C)

```
Animal animal = Cachorro();  
animal.emitirSom();
```

D)

```
String nome = 'Ana';  
print(nome);
```

E)

```
double salario = 1500.00;
```

Questão 36. Observe o código:

```
class Calculadora {  
    int somar(int a, int b) {  
        return a + b;  
    }  
  
    int somar(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Qual é o problema?

- A) Dart não permite métodos com retorno int.
- B) Dart não permite sobrecarga tradicional de métodos com o mesmo nome e assinaturas diferentes.
- C) O método somar() deveria ser abstrato.
- D) A classe Calculadora deveria herdar de outra classe.
- E) O segundo método deveria usar @override.

Questão 37. Analise o cenário:

Um sistema possui as classes Funcionario, Professor, Coordenador e Estagiario. Todas têm o método calcularPagamento(), mas cada uma calcula de forma diferente. O sistema deve percorrer uma lista de funcionários e calcular o pagamento de todos.

Qual solução usa melhor o polimorfismo?

- A) Criar uma lista de String com os nomes das classes.
- B) Criar uma lista List<Funcionario> e fazer cada classe filha sobrescrever calcularPagamento().
- C) Criar uma classe separada para cada cálculo e não relacionar nenhuma delas.
- D) Usar vários if no main para verificar manualmente cada tipo de funcionário.
- E) Criar todos os métodos como static.

Questão 38. Observe o código:

```
abstract class Pagamento {  
    void pagar(double valor);  
}  
  
class PagamentoPix implements Pagamento {  
    @override  
    void pagar(double valor) {  
        print('Pix: R$ $valor');  
    }  
}  
  
class PagamentoCartao implements Pagamento {  
    @override  
    void pagar(double valor) {  
        print('Cartão: R$ $valor');  
    }  
}
```

```
void processarPagamento(Pagamento pagamento) {  
    pagamento.pagar(100);  
}  
  
void main() {  
    processarPagamento(PagamentoPix());  
    processarPagamento(PagamentoCartao());  
}
```

Qual conceito é demonstrado no código?

- A) Encapsulamento sem polimorfismo.
- B) Polimorfismo por interface.
- C) Herança múltipla inválida.
- D) Sobrecarga de métodos.
- E) Recursão.

Sobrecarga de operadores

Questão 39. Em Dart, qual alternativa declara corretamente a sobrecarga do operador +?

A)

```
class Vetor {  
    Vetor +(Vetor outro) {  
        return outro;  
    }  
}
```

B)

```
class Vetor {  
    Vetor operator +(Vetor outro) {  
        return outro;  
    }  
}
```

C)

```
class Vetor {  
    operator Vetor +(Vetor outro) {  
        return outro;  
    }  
}
```

D)

```
class Vetor {  
    overload +(Vetor outro) {  
        return outro;  
    }  
}
```

E)

```
class Vetor {  
    Vetor plus operator(Vetor outro) {  
        return outro;  
    }  
}
```


Questão 40. Analise as afirmativas sobre sobrecarga de operadores em Dart:

- I. Dart permite sobrecarregar alguns operadores.
 - II. A sobrecarga deve ser feita dentro de uma classe.
 - III. O operador + pode ser sobrecarregado com o uso de operator +.
 - IV. O operador = pode ser sobrecarregado em Dart.
- A) Apenas I e II estão corretas.
B) Apenas I, II e III estão corretas.
C) Apenas II, III e IV estão corretas.
D) Apenas I e IV estão corretas.
E) Todas estão corretas.

Questão 41. Observe o código:

```
class Caixa {  
    List<String> itens = ['Caneta', 'Lápis', 'Borracha'];  
  
    String operator [](int indice) {  
        return itens[indice];  
    }  
}  
  
void main() {  
    Caixa caixa = Caixa();  
  
    print(caixa[1]);  
}
```

Qual será a saída?

- A) Caneta
B) Lápis
C) Borracha
D) 1
E) Erro, pois [] não pode ser sobrecarregado.

Questão 42. Complete corretamente o código:

```
class Temperatura {  
    double celsius;  
  
    Temperatura(this.celsius);  
  
    bool _____(Temperatura outra) {  
        return celsius > outra.celsius;  
    }  
}
```

Para permitir o uso de:

```
Temperatura t1 = Temperatura(30);
```

```
Temperatura t2 = Temperatura(20);  
  
print(t1 > t2);
```

- A) maiorQue
- B) operator >
- C) override >
- D) extends >
- E) compare >

Questão 43. Observe o código:

```
class Dinheiro {  
    double valor;  
  
    Dinheiro(this.valor);  
  
    Dinheiro operator +(Dinheiro outro) {  
        return Dinheiro(valor + outro.valor);  
    }  
}  
  
void main() {  
    Dinheiro d1 = Dinheiro(10);  
    Dinheiro d2 = Dinheiro(15);  
  
    Dinheiro total = d1 + d2;  
  
    print(total.valor);  
}
```

Qual será a saída?

- A) 10
- B) 15
- C) 25
- D) 1015
- E) Erro, pois + só funciona com números primitivos.

Questão 44. Em Programação Orientada a Objetos, sobrecarga de operadores é:

- A) A criação de vários construtores com nomes diferentes.
- B) A redefinição do comportamento de operadores, como +, - ou ==, para objetos de uma classe.
- C) A obrigação de toda classe herdar de outra classe.
- D) A criação de métodos abstratos sem corpo.
- E) A transformação automática de uma classe em interface.

Questão 45. Analise o código:

```
class Ponto {  
    int x;
```

```

int y;

Ponto(this.x, this.y);

@Override
bool operator ==(Object outro) {
    if (outro is Ponto) {
        return x == outro.x && y == outro.y;
    }

    return false;
}
}

```

Qual é a principal melhoria necessária nesse código?

- A) Remover o operador ==, pois ele não pode ser sobrescrito.
- B) Alterar o retorno de bool para int.
- C) Sobrescrever também hashCode.
- D) Transformar Ponto em classe abstrata.
- E) Trocar Object por String.

Manipulação de arquivos

Questão 46. Observe o código:

```

import 'dart:io';

void main() async {
    File arquivo = File('nomes.txt');

    await arquivo.writeAsString('Ana\n');
    await arquivo.writeAsString('Bruno\n');

    String conteudo = await arquivo.readAsString();

    print(conteudo);
}

```

Qual será o conteúdo final do arquivo?

A)

```

Ana
Bruno

```

B)

```

Ana

```

C)

```

Bruno

```

D)

```

AnaBruno

```

E) O arquivo ficará vazio.

Questão 47. Observe o código:

```
import 'dart:io';

class AlunoDao {
  File arquivo = File('alunos.txt');

  Future<void> salvar(String nome, int idade) async {
    await arquivo.writeAsString('$nome;$idade\n', mode: FileMode.append);
  }
}
```

Qual responsabilidade principal da classe AlunoDao?

- A) Controlar a interface gráfica do sistema.
- B) Representar uma estrutura de repetição.
- C) Encapsular a operação de persistência dos alunos no arquivo.
- D) Calcular a média final dos alunos.
- E) Criar uma herança múltipla entre alunos.

Questão 48. Observe o código:

```
import 'dart:io';

void main() async {
  try {
    File arquivo = File('inexistente.txt');
    String conteudo = await arquivo.readAsString();

    print(conteudo);
  } catch (erro) {
    print('Erro ao ler o arquivo');
  }
}
```

Qual é a finalidade do bloco try/catch nesse código?

- A) Repetir a leitura do arquivo várias vezes.
- B) Impedir que o arquivo seja criado.
- C) Tratar uma possível exceção durante a leitura do arquivo.
- D) Transformar o arquivo em uma lista.
- E) Fazer a leitura ficar síncrona.

Questão 49. Analise o cenário:

Um sistema precisa salvar alunos em um arquivo texto. Cada aluno possui nome e idade. Cada linha do arquivo deve seguir o formato:

```
nome;idade
```

Qual solução é mais adequada?

A)

```
class Aluno {
  String nome;
  int idade;
```

```

    Aluno(this.nome, this.idade);

    String paraLinha() {
        return '$nome;$idade';
    }
}

```

B)

```

class Aluno {
    String nome;
    int idade;

    Aluno(this.nome, this.idade);

    void paraLinha() {
        print(nome);
        print(idade);
    }
}

```

C)

```

class Aluno {
    String nome;
    int idade;

    Aluno(this.nome, this.idade);

    int paraLinha() {
        return idade;
    }
}

```

D)

```

class Aluno {
    String nome;
    int idade;

    Aluno(this.nome, this.idade);

    File paraLinha() {
        return File(nome);
    }
}

```

E)

```

class Aluno {
    String nome;
    int idade;

    Aluno(this.nome, this.idade);

    bool paraLinha() {
        return true;
    }
}

```

Questão 50. Observe o código:

```
import 'dart:io';

void main() async {
  File arquivo = File('mensagem.txt');

  String conteudo = await arquivo.readAsString();

  print(conteudo);
}
```

O que esse código faz?

- A) Escreve uma mensagem no arquivo.
- B) Remove o arquivo do computador.
- C) Lê o conteúdo do arquivo como texto e imprime na tela.
- D) Cria uma nova pasta.
- E) Abre o arquivo em modo binário.

Exceções

Questão 51. Em programação, uma exceção é:

- A) Um erro ou situação inesperada que pode interromper o fluxo normal do programa.
- B) Uma classe que obrigatoriamente herda de outra classe.
- C) Um operador usado apenas para somar objetos.
- D) Uma forma de criar listas dinâmicas.
- E) Um método que sempre retorna verdadeiro.

Questão 52. Qual estrutura é usada em Dart para tentar executar um bloco de código e tratar possíveis exceções?

- A) if / else
- B) for / while
- C) try / catch
- D) class / extends
- E) return / break

Questão 53. Observe o código:

```
void main() {
  try {
    int numero = int.parse('abc');
    print(numero);
  } catch (erro) {
    print('Valor inválido');
  }
}
```

Qual será a saída?

- A) abc
- B) 0
- C) Valor inválido
- D) null
- E) O código não compila porque try não pode conter variáveis.

Questão 54. Analise as afirmativas sobre tratamento de exceções em Dart:

- I. O bloco try contém o código que pode gerar uma exceção.
 - II. O bloco catch pode capturar e tratar uma exceção.
 - III. O bloco finally executa apenas quando ocorre erro.
 - IV. O comando throw pode lançar uma exceção manualmente.
- A) Apenas I e II estão corretas.
 - B) Apenas I, II e IV estão corretas.
 - C) Apenas II, III e IV estão corretas.
 - D) Apenas III e IV estão corretas.
 - E) Todas estão corretas.

Questão 55. Complete corretamente o código para lançar uma exceção quando o saque for maior que o saldo:

```
class Conta {  
    double saldo = 100;  
  
    void sacar(double valor) {  
        if (valor > saldo) {  
            _____ Exception('Saldo insuficiente');  
        }  
  
        saldo -= valor;  
    }  
}
```

- A) try
- B) catch
- C) throw
- D) finally
- E) error

Questão 56. Observe o código:

```
void main() {  
    try {  
        throw Exception('Erro no cadastro');  
    } catch (erro) {  
        print('Erro tratado');
```

```
    } finally {  
        print('Finalizando');  
    }  
}
```

Qual será a saída?

- A) Erro tratado
- B) Finalizando
- C) Erro tratado
Finalizando
- D) Erro no cadastro
- E) Nenhuma saída.

Questão 57. Observe o código:

```
void validarIdade(int idade) {  
    if (idade < 0) {  
        throw Exception('Idade inválida');  
    }  
}  
  
void main() {  
    validarIdade(-5);  
    print('Cadastro realizado');  
}
```

Qual é o principal problema desse código?

- A) O método validarIdade deveria retornar String.
- B) A exceção é lançada, mas não há try/catch para tratá-la no main.
- C) O if não pode comparar números inteiros.
- D) O throw só pode ser usado dentro de classes abstratas.
- E) O print deveria estar antes da validação.

Questão 58. Qual alternativa trata corretamente a exceção do código da questão anterior?

A)

```
void main() {  
    try {  
        validarIdade(-5);  
        print('Cadastro realizado');  
    } catch (erro) {  
        print('Erro ao cadastrar');  
    }  
}
```

B)

```
void main() {  
    catch {  
        validarIdade(-5);  
    }  
}
```



```
}
```

C)

```
void main() {  
    validarIdade(-5) catch (erro);  
}
```

D)

```
void main() {  
    throw validarIdade(-5);  
}
```

E)

```
void main() {  
    finally {  
        validarIdade(-5);  
    }  
}
```

MVC e DAO

Questão 59. No padrão MVC, a camada Model é responsável por:

- A) Exibir menus e mensagens para o usuário.
- B) Representar os dados e regras principais do sistema.
- C) Controlar diretamente o teclado do usuário.
- D) Substituir todas as classes DAO.
- E) Compilar o programa automaticamente.

Questão 60. No padrão MVC, qual é a função principal do Controller?

- A) Receber ações da View, coordenar o fluxo e acionar Model e DAO quando necessário.
- B) Armazenar diretamente todos os dados em arquivos.
- C) Servir apenas para imprimir textos na tela.
- D) Representar exclusivamente uma tabela do banco.
- E) Impedir a criação de objetos.

Questão 61. Em uma aplicação com DAO, a principal responsabilidade de uma classe DAO é:

- A) Representar visualmente os botões do sistema.
- B) Concentrar operações de acesso a dados, como salvar, listar, buscar, alterar e remover.
- C) Substituir a necessidade de classes Model.
- D) Criar menus no terminal.
- E) Definir herança múltipla entre entidades.

Questão 62. Analise as afirmativas sobre MVC e DAO:

- I. A View deve ser responsável pela interação com o usuário.
- II. O Controller deve coordenar a comunicação entre View, Model e DAO.

III. O DAO deve concentrar operações de persistência.

IV. A classe Model deve obrigatoriamente imprimir menus no console.

A) Apenas I e II estão corretas.

B) Apenas I, II e III estão corretas.

C) Apenas II, III e IV estão corretas.

D) Apenas III e IV estão corretas.

E) Todas estão corretas.

Questão 63. Observe o código:

```
class Livro {
    String titulo;
    String autor;

    Livro(this.titulo, this.autor);

    Future<void> salvarNoArquivo() async {
        // grava o livro diretamente no arquivo
    }
}
```

Qual é o principal problema de organização?

A) A classe Livro não pode ter atributos.

B) A classe Livro mistura a responsabilidade de entidade/modelo com persistência de dados.

C) O construtor deveria ser abstrato.

D) O método salvarNoArquivo deveria ser obrigatório em toda classe.

E) O atributo autor deveria ser int.

Questão 64. Qual alternativa representa uma melhor separação entre Model e DAO?

A)

```
class Livro {
    String titulo;
    String autor;

    Livro(this.titulo, this.autor);
}

class LivroDao {
    Future<void> salvar(Livro livro) async {
        // salva o livro
    }
}
```

B)

```
class Livro {
    Future<void> salvar() async {
        // salva no arquivo
    }
}
```

```
}
```

C)

```
class LivroView {  
    String titulo;  
    String autor;  
}
```

D)

```
class LivroController extends LivroDao, Livro {  
}
```

E)

```
void main() {  
    // todo o sistema fica apenas no main  
}
```

Questão 65. Complete corretamente o código para que o Controller use o DAO para cadastrar um livro:

```
class LivroController {  
    LivroDao dao = LivroDao();  
  
    Future<void> cadastrar(String titulo, String autor) async {  
        Livro livro = Livro(titulo, autor);  
        await _____.salvar(livro);  
    }  
}
```

A) dao

B) Livro

C) View

D) main

E) extends

Questão 66. Observe o cenário:

Um sistema de biblioteca possui cadastro de livros, listagem de livros disponíveis, cadastro de usuários, empréstimo e devolução. Deseja-se organizar o sistema usando MVC e DAO.

Qual organização é mais adequada?

A) Colocar todas as regras, menus e gravações de arquivo dentro do main.

B) Criar Models como Livro e Usuario, DAOs como LivroDao e UsuarioDao, Controllers para coordenar as operações e Views para interação com o usuário.

C) Criar apenas uma classe View responsável por salvar arquivos e calcular todas as regras.

D) Criar uma classe Livro herdando de Usuario para reaproveitar código.

E) Usar DAO apenas para imprimir mensagens no terminal.